CLASE 5 · MÓDULO 4

Redes multicapa y retropropagación

Capas ocultas, pase hacia adelante, retropropagación… ¡y XOR resuelto!

github.com/datahuba/curso-redes-neuronales · Basado en el curso del Dr. J. Irving Vasquez (jivg.org)

Agenda de hoy

Tiempo	Tema
0:00	Repaso de la Clase 4
0:10	¿Por qué capas ocultas?
0:25	Pesos como matrices y pase hacia adelante
0:45	Práctica: 1_09
1:05	Descanso
1:15	Retropropagación
1:45	Práctica: 1_10
2:00	Entrenar la red completa y resolver XOR
2:20	Quiz y cierre

Al terminar la clase podrás…

1

Explicar

por qué una capa oculta resuelve XOR.

2

Calcular

el pase hacia adelante con matrices.

3

Aplicar

un paso de retropropagación a mano.

4

Entrenar

una red multicapa en Python.

Repaso de la Clase 4

¿Regla delta?

Término de error, incremento y actualización.

¿Varias salidas?

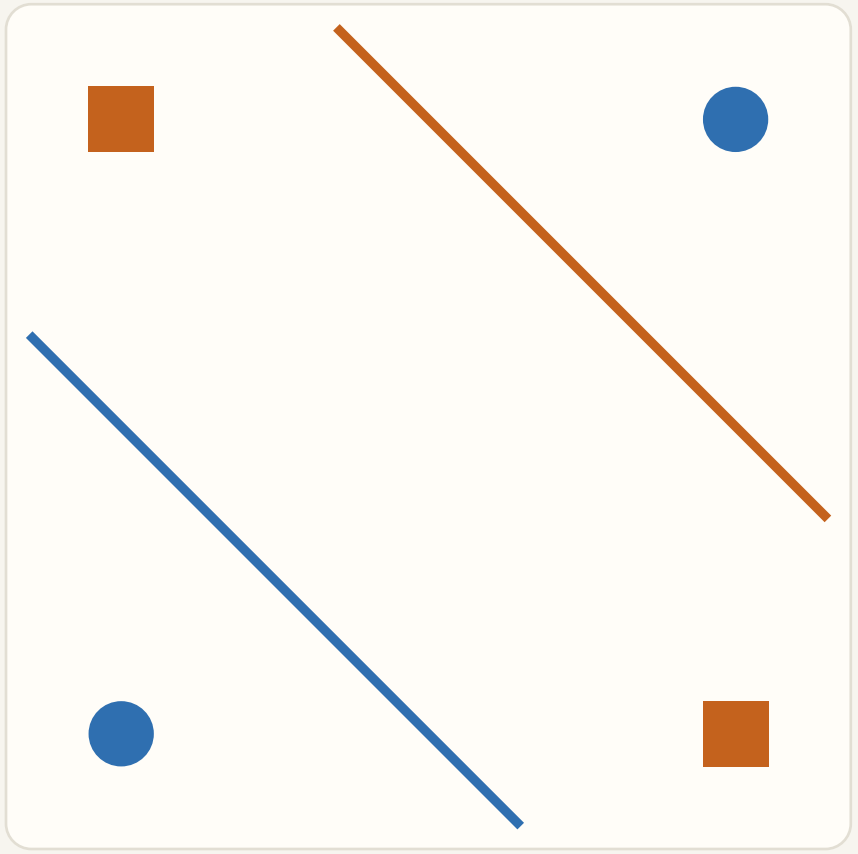
H = X·W + B: una matriz de pesos.

¿Qué quedó pendiente?

XOR: una neurona solo traza una recta.

Hoy: juntar neuronas en capas para resolver XOR.

XOR con dos rectas



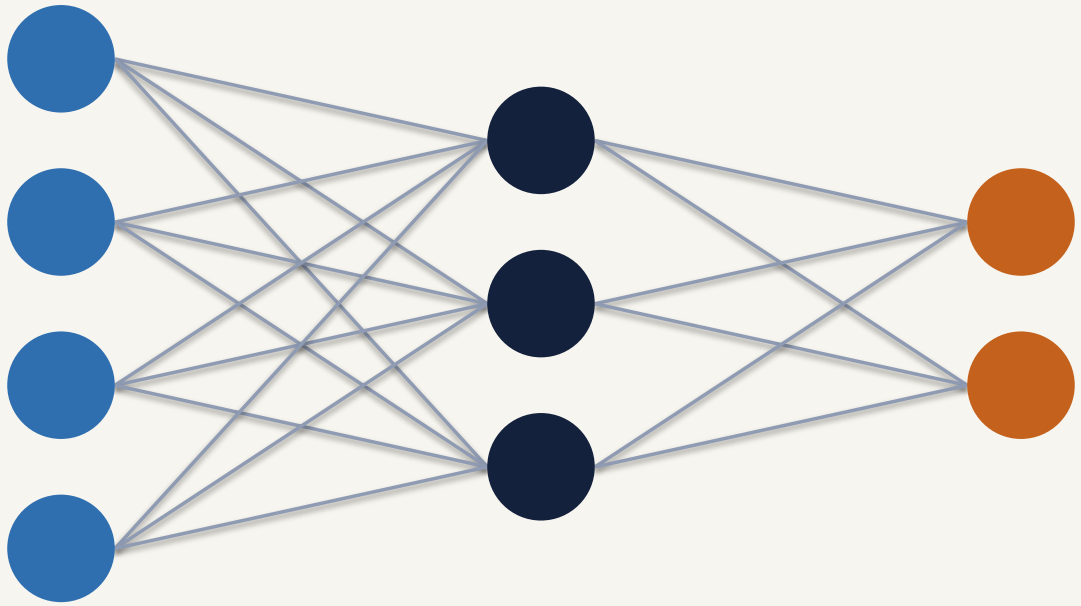
Neurona oculta 1: recta A (deja abajo solo el (0, 0)).

Neurona oculta 2: recta B (deja arriba solo el (1, 1)).

Neurona de salida: “arriba de A y debajo de B”.

Cada neurona oculta traza una recta; la salida las combina en una franja.

Perceptrón multicapa (MLP)



Entrada (4) **Oculta (3)** **Salida (2)**

Entrada: los datos; no calcula nada.

Oculta: su salida no se ve desde afuera.

Salida: la predicción.

Muchas capas ocultas = aprendizaje profundo.

Los pesos son matrices

Matriz	Conecta	Tamaño
W_1	entrada (4) → oculta (3)	4 × 3
W_2	oculta (3) → salida (2)	3 × 2
B_1	un sesgo por neurona oculta	1 × 3
B_2	un sesgo por neurona de salida	1 × 2

$$(a \times b) \cdot (b \times c) = (a \times c)$$

el número del medio debe coincidir

Ejemplo:

$$(1 \times 4) \cdot (4 \times 3) = (1 \times 3)$$

una fila de 3 números: las 3 neuronas ocultas.

Pase hacia adelante

Capa oculta

$$H_1 = X \cdot W_1 + B_1$$

$$A_1 = \sigma(H_1)$$

tamaño: 1 × 3

Capa de salida

$$H_2 = A_1 \cdot W_2 + B_2$$

$$\hat{Y} = \sigma(H_2)$$

tamaño: 1 × 2

La salida de una capa (A_1) es la entrada de la siguiente.

Contar parámetros

Capa	Pesos	Sesgos	Total
4 → 3	4·3 = 12	3	15
3 → 2	3·2 = 6	2	8
Red 4 → 3 → 2	18	5	**23**

Regla por capa:

`(entradas + 1) × neuronas`

NOTEBOOKS · 20 MIN · 3 PUNTOS

Práctica · 1_09_red_multicapa_ejercicio

TODO 1 pt tamaños 4, 3, 2 y matrices W_1, W_2 con np.random.normal

TODO 1 pt pase hacia adelante con np.dot

1 pt agregar sesgos B_1 (1×3) y B_2 (1×2)

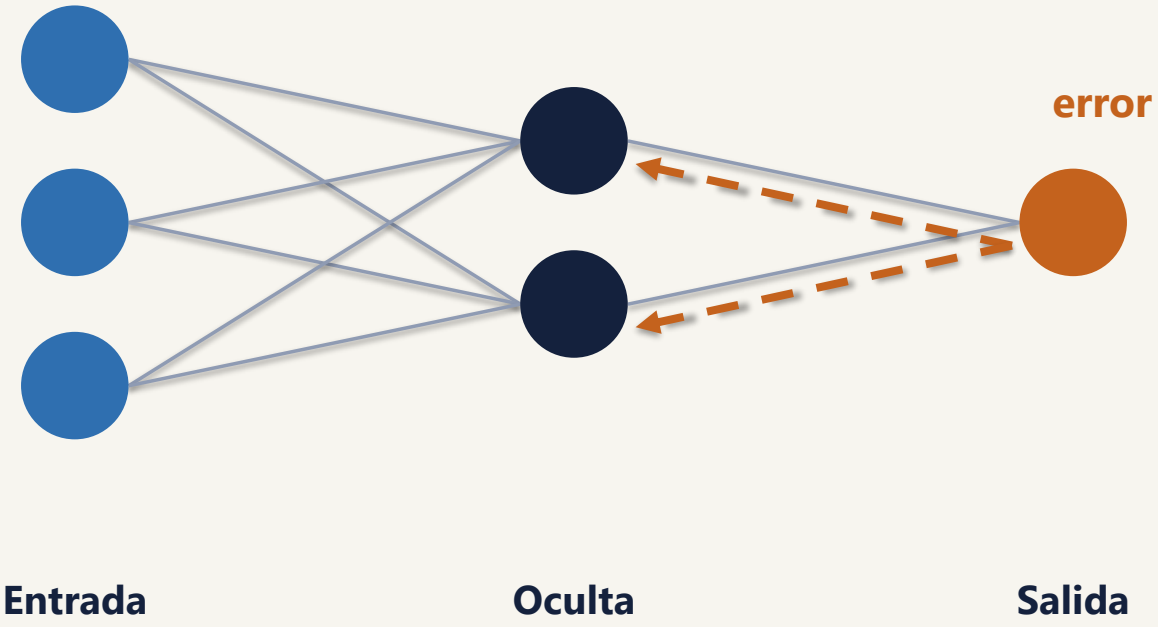
Pista: la segunda capa usa A_1 (después de la sigmoide), no H_1. Desviación sugerida: 1/√4.

```
# comprobaciones
W_1: (4, 3)  W_2: (3, 2)  X: (1, 4)
oculta: 3 números entre 0 y 1
salida: 2 números entre 0 y 1
parámetros con sesgos: 23
```

Descanso · 10 min

Después: la retropropagación

¿Cuál es el error de una neurona oculta?



Nadie nos dice qué debió producir una neurona oculta.

Solución: calcular el error en la salida y **mandarlo hacia atrás**.

Cada neurona oculta recibe una parte según el peso que la une con la salida.

Las fórmulas

Capa	Término de error	Incremento de sus pesos
Salida	$\delta_o = (y - \hat{y}) \cdot \hat{y}(1 - \hat{y})$	$\Delta w_ho = \eta \cdot \delta_o \cdot a_h$
Oculta	$\delta_h = \delta_o \cdot w_ho \cdot a_h(1 - a_h)$	$\Delta w_ih = \eta \cdot \delta_h \cdot x_i$

a_h = salida de la neurona oculta.

La oculta usa la regla delta, pero su "error" es **δ_o × el peso que la une con la salida**.

Ejemplo (notebook 1_10): hacia adelante

`x = [0.5, 0.1, -0.2] · y = 0.6 · η = 0.5 · W_ih = [[0.5, -0.6], [0.1, -0.2], [0.1, 0.7]] · W_ho = [0.1, -0.3]`

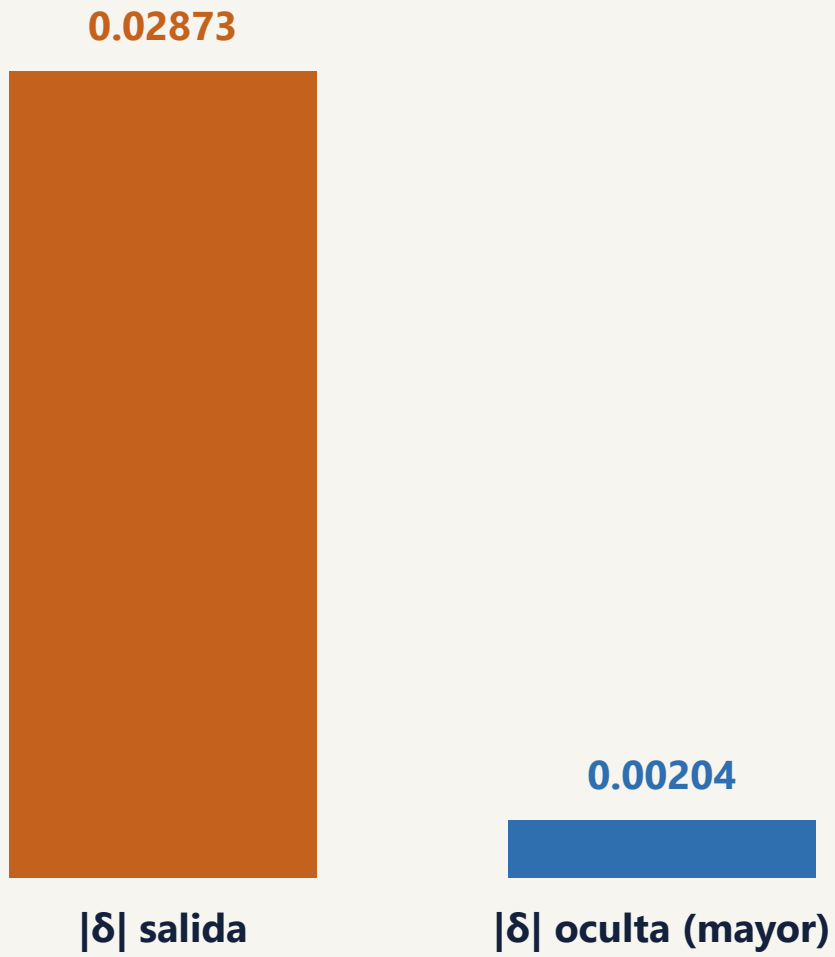
Valor	Cálculo	Resultado
Entrada a la oculta	x · W_ih	[0.24, −0.46]
Salida oculta a_h	σ(…)	[0.5597, 0.3870]
Entrada a la salida	0.5597·0.1 + 0.3870·(−0.3)	−0.0601
Salida ŷ	σ(−0.0601)	**0.4850**

Ejemplo (notebook 1_10): hacia atrás

Valor	Cálculo	Resultado
Error	$0.6 - 0.4850$	0.1150
δ_o	$0.1150 \cdot 0.4850 \cdot 0.5150$	0.02873
δ_h	$0.02873 \cdot [0.1, -0.3] \cdot a_h(1 - a_h)$	$[0.000708, -0.002045]$
Δw_ho	$0.5 \cdot 0.02873 \cdot [0.5597, 0.3870]$	$[0.00804, 0.00556]$
Δw_ih	$0.5 \cdot \delta_h \cdot x$ (matriz 3×2)	$[[0.000177, -0.000511], \ldots]$

Cada peso tiene su incremento: las matrices de incrementos miden lo mismo que las de pesos.

El gradiente que se desvanece



La capa oculta recibe un error ~14 veces más pequeño.

Se multiplica por pesos pequeños y por derivadas de la sigmoide (≤ 0.25).

Con muchas capas, las primeras casi no aprenden.

Por eso las redes profundas usan ReLU.

NOTEBOOKS · 15 MIN · 3 PUNTOS

Práctica · 1_10_retropropagacion_base

TODO 1 pt error, δ de salida y δ de la oculta

TODO 1 pt incrementos y actualización

1 pt ¿mismo tamaño que los pesos?

Pista: para Δw_ih convierte x en columna con x[:, None]: da una matriz 3 × 2.

```
# comprobaciones (η = 0.5)
error = 0.1150
delta salida = 0.02873
delta oculta = [0.000708 -0.002045]
Δw_ho = [0.00804 0.00556]
```

Entrenar la red completa (notebook 1_11)

1. Incrementos acumulados = 0
2. Por ejemplo: adelante → δ_o y δ_h → acumular
3. Actualizar W_ih y W_ho
4. Medir la pérdida y repetir

Actualización	Pérdida	Exactitud
0.001 · acumulado	0.265 → 0.201	0.725
0.5 · acum. / n	0.264 → 0.200	**0.750**

Ojo: el notebook original imprime la pérdida con solo el último ejemplo (x). Usa features.values.

NOTEBOOKS · 20 MIN · 3 PUNTOS · + DOCS/PRACTICA_05.PY

Práctica · 1_11 + ¡XOR resuelto!

TODO 1 pt ciclo de entrenamiento

TODO 1 pt curva de aprendizaje

1 pt preguntas

Demo: python docs/practica_05.py --graficar. La frontera aprendida ya no es una recta.

```
# python docs/practica_05.py  (red 2-3-1)
[0 0] -> 0.025 -> 0
[0 1] -> 0.981 -> 1
[1 0] -> 0.971 -> 1
[1 1] -> 0.026 -> 0
pérdida: 0.2596 -> 0.0006
```

Quiz · 5 minutos

1. ¿Por qué una capa oculta permite resolver XOR?
2. ¿Qué tamaño tiene W entre una capa de 4 y una de 3?
3. ¿Cuántos parámetros tiene una red 2 → 3 → 1?
4. ¿Cómo se calcula el error de una neurona oculta?
5. ¿Qué es el gradiente que se desvanece?

Para llevar

1. Las capas ocultas combinan rectas: resuelven problemas no lineales.
2. Pase hacia adelante: la clase 4 repetida capa por capa.
3. Retropropagación: el error viaja hacia atrás, repartido según los pesos.

Próxima clase: optimizadores (cómo caminar mejor por la montaña del error).

Tarea: notebooks 1_09, 1_10 y 1_11 (9 pts)

Leer: libro, “Retropropagación”